

L17 — Traversal and Searching in a Linked List

Unit: 2 | Chapter: Linked Lists | BT Level: BT3 | CO: CO2, CO5

Learning Objectives

- Implement forward and backward traversal of linked lists
 - Search for an element in a singly linked list
 - Apply the two-pointer (slow-fast) technique for special traversal problems
 - Analyze traversal and search complexity
-

1. Traversal of a Singly Linked List

Traversal visits each node exactly once, starting from `HEAD` and following `next` pointers until `NULL`.

```
def traverse(head):  
    current = head  
    while current is not None:  
        print(current.data, end=" → ")  
        current = current.next  
    print("NULL")
```

Time: $O(n)$ — must visit every node

Space: $O(1)$ — only one pointer variable used

Key Pattern: The Traversal Template

Almost all linked list operations follow this template:

```
current = head  
while current is not None:  
    # Do something with current.data  
    current = current.next
```

2. Computing List Properties via Traversal

2.1 Length

```
def length(head):  
    count = 0  
    current = head  
    while current:  
        count += 1
```

```
        current = current.next
    return count    # O(n)
```

2.2 Sum and Maximum

```
def sum_and_max(head):
    total = 0
    max_val = float('-inf')
    current = head
    while current:
        total += current.data
        if current.data > max_val:
            max_val = current.data
        current = current.next
    return total, max_val    # O(n), O(1) space
```

2.3 Count Occurrences

```
def count_occurrences(head, key):
    count = 0
    current = head
    while current:
        if current.data == key:
            count += 1
        current = current.next
    return count    # O(n)
```

3. Searching in a Linked List

3.1 Linear Search — O(n)

No binary search is possible on a linked list (no random access by index).

```
def search(head, key):
    """
    Returns the index (0-based) of the first occurrence of key,
    or -1 if not found.
    """
    current = head
    index = 0
    while current:
        if current.data == key:
            return index
        current = current.next
        index += 1
    return -1
```

3.2 Search and Return Node Reference

```
def search_node(head, key):
    """Returns the node containing key, or None if not found."""
```

```
current = head
while current:
    if current.data == key:
        return current
    current = current.next
return None
```

4. Accessing the k-th Node

Since linked lists have no index, accessing the k-th element requires traversal:

```
def get_kth(head, k):
    """
    Returns the k-th node (0-based index).
    Returns None if k >= length.
    """
    current = head
    for _ in range(k):
        if current is None:
            return None
        current = current.next
    return current    # O(k)
```

Finding the k-th from the end using two pointers:

```
def kth_from_end(head, k):
    """
    Returns the k-th node from the end (1-based).
    Uses two pointers separated by k positions.
    """
    fast = head
    slow = head

    # Move fast k steps ahead
    for _ in range(k):
        if fast is None:
            return None    # k > length
        fast = fast.next

    # Move both until fast reaches end
    while fast:
        fast = fast.next
        slow = slow.next

    return slow    # O(n), O(1) space
```

5. Two-Pointer (Slow-Fast) Technique

This is a powerful technique for linked list problems.

5.1 Finding the Middle Node

```
def find_middle(head):  
    """  
    Uses slow and fast pointers.  
    Slow moves 1 step, fast moves 2 steps.  
    When fast reaches end, slow is at middle.  
    """  
    if head is None:  
        return None  
    slow = head  
    fast = head  
    while fast and fast.next:  
        slow = slow.next          # +1 step  
        fast = fast.next.next     # +2 steps  
    return slow    # O(n), O(1) space
```

Trace for [1→2→3→4→5]:

Step	slow	fast
Start	1	1
1	2	3
2	3	5
3	—	fast.next=NULL, stop

slow = 3 (middle) ✓

5.2 Detecting a Cycle (Floyd's Cycle Detection)

```
def has_cycle(head):  
    """  
    If slow and fast ever meet, there is a cycle.  
    Time: O(n), Space: O(1)  
    """  
    slow = head  
    fast = head  
    while fast and fast.next:  
        slow = slow.next  
        fast = fast.next.next  
        if slow == fast:    # Pointers met → cycle exists  
            return True  
    return False
```

6. Traversal in Reverse (Without Reversing)

To print a singly linked list in reverse order without reversing:

```
def print_reverse(head):  
    """Recursive — O(n) time, O(n) space (call stack)"""  
    if head is None:
```

```
        return
    print_reverse(head.next)    # Go to end first
    print(head.data, end=" ")  # Then print on the way back
```

7. Comparison: Array vs Linked List Search

Feature	Array	Linked List
Sorted data search	$O(\log n)$ binary search	$O(n)$ — no random access
Unsorted search	$O(n)$	$O(n)$
Access by index	$O(1)$	$O(n)$
Membership check	$O(n)$ unsorted, $O(\log n)$ sorted	$O(n)$

Linked lists are inferior to arrays for pure searching. Their advantage lies in **insertion/deletion**, not search.

Summary

- Traversal follows `HEAD → node → node → ... → NULL`, visiting each node once: **$O(n)$** .
 - Searching requires linear scan (no binary search possible): **$O(n)$** .
 - The **two-pointer technique** solves many linked list problems in $O(n)$ with $O(1)$ space: middle node, k -th from end, cycle detection.
 - Printing in reverse uses recursion: $O(n)$ time and space.
-

References

- Cormen et al., *Introduction to Algorithms*, Ch. 10.2 (MIT Press, 2022)
- Skiena, *The Algorithm Design Manual*, Ch. 3.1 (Springer, 2020)
- LeetCode: #141 (Cycle Detection), #876 (Middle of List)